



Basic Information

- Experiment No: 05
- Date: 14/09/2026
- Title: Demonstration of OOP Concepts Using a Library Management Example



Objective / Aim

To write a Java program that demonstrates the major concepts of Object-Oriented Programming:

1. Encapsulation
2. Inheritance
3. Method overloading
4. Method overriding
5. Abstraction
6. Interfaces



Classes and objects

System Requirements

Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM
- Keyboard and monitor

Software Requirements

- Java Development Kit (JDK)
- IntelliJ IDEA / Eclipse / NetBeans



Windows, Linux, or macOS operating system

Algorithm / Logic Description

1. Start the program.
2. Define the Book class.
3. Declare the following private data members in Book:
 - a. bookId
 - b. bookName
 - c. author
 - d. price
4. Define a parameterized constructor in Book.
5. Initialize all Book data members using the constructor.
6. Define getter and setter methods for all private data members.
7. Define the display() method to print book details.
8. Define the Person class with:
 - a. name
 - b. age
 - c. display() method
9. Define the Student class that extends Person.
10. Add the course variable to Student.
11. Override the display() method in Student.
12. Use super.display() to display the parent class details.
13. Display the student's course.
14. Define the Faculty class that extends Person.
15. Add the department variable to Faculty.
16. Override the display() method in Faculty.

17. Use `super.display()` to display the parent class details.
18. Display the faculty department.
19. Define the Area class.
20. Create an overloaded `area()` method to calculate the area of a circle.
21. Create another overloaded `area()` method to calculate the area of a rectangle.
22. Define the Vehicle class with a `display()` method.
23. Define the Car class that extends Vehicle.
24. Override the `display()` method in Car.
25. Define the Bike class that extends Vehicle.
26. Override the `display()` method in Bike.
27. Define the abstract Shape class with an abstract `draw()` method.
28. Define the Circle class that extends Shape.
29. Implement the `draw()` method in Circle.
30. Define the Rectangle class that extends Shape.
31. Implement the `draw()` method in Rectangle.
32. Define the Printable interface with the `print()` method.
33. Define the Report class that implements Printable.
34. Implement the `print()` method in Report.
35. In the `main()` method, create a Book object.
36. Display the book details.
37. Create a Student object and assign its details.
38. Display the student details.

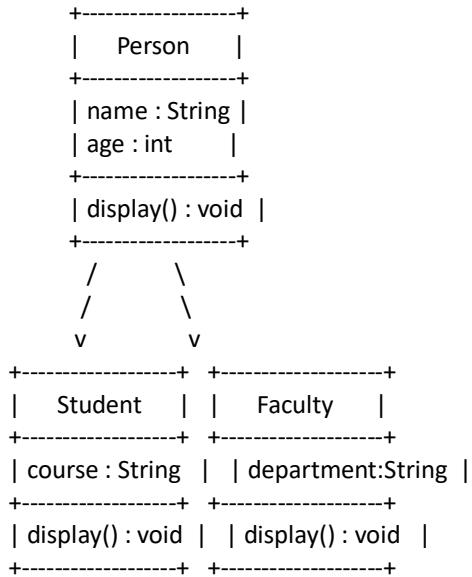
39. Create a Faculty object and assign its details.
40. Display the faculty details.
41. Create an Area object.
42. Call both overloaded area() methods.
43. Create Vehicle references pointing to Car and Bike objects.
44. Call the overridden display() methods.
45. Create Shape references pointing to Circle and Rectangle objects.
46. Call the draw() methods.
47. Create a Printable reference pointing to a Report object.
48. Call the print() method.
49. Stop the program.



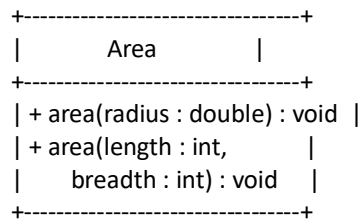
UML Class Diagram

+-----+	
	Book
+-----+	
	- bookId : int
	- bookName : String
	- author : String
	- price : double
+-----+	
	+ Book(id, name, auth, pr)
	+ getBookId() : int
	+ setBookId(id : int) : void
	+ getBookName() : String
	+ setBookName(name : String) : void
	+ getAuthor() : String
	+ setAuthor(auth : String) : void
	+ getPrice() : double
	+ setPrice(pr : double) : void
	+ display() : void
+-----+	

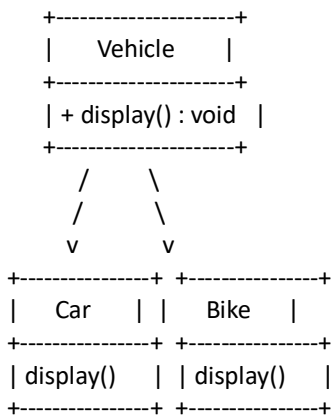
Inheritance



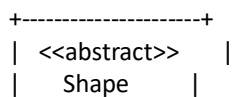
Method Overloading

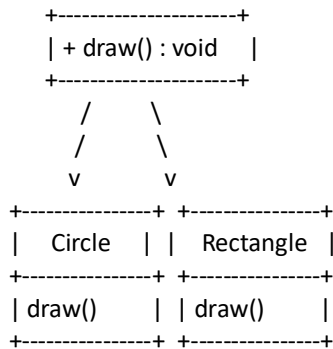


Method Overriding

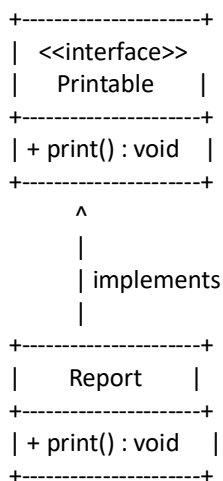


Abstraction





Interface



SOURCE CODE:

```

class Book {
    private int bookId;
    private String bookName;
    private String author;
    private double price;
    Book(int id, String name, String auth, double pr) {
        bookId = id;
        bookName = name;
        author = auth;
        price = pr;
    }
    public int getBookId() {
        return bookId;
    }
    public void setBookId(int id) {

```

```

        bookId = id;
    }
    public String getBookName() {
        return bookName;
    }
    public void setBookName(String name) {
        bookName = name;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String auth) {
        author = auth;
    }
    public double getPrice() {
        return price;
    }
    public void setPrice(double pr) {
        price = pr;
    }
    public void display() {
        System.out.println("Book ID : " + bookId);
        System.out.println("Book Name : " + bookName);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

```

// Part C: Inheritance

```

class Person {
    String name;
    int age;
    void display() {
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
    }
}
class Student extends Person {
    String course;
    void display() {
        super.display();
        System.out.println("Course : " + course);
    }
}
class Faculty extends Person {
    String department;
    void display() {
        super.display();
    }
}

```

```

        System.out.println("Department : " + department);
    }
}
// Part D: Method Overloading
class Area {
    void area(double radius) { // circle area
        System.out.println("Circle Area = " + (3.14 * radius * radius));
    }
    void area(int length, int breadth) { // rectangular area
        System.out.println("Rectangle Area = " + (length * breadth));
    }
}
// Part D: Method Overriding
class Vehicle {
    void display() {
        System.out.println("This is a vehicle.");
    }
}
class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a car.");
    }
}
class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a bike.");
    }
}
// Part E: Abstraction
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}
class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}
// Part E: Interface

```

```

interface Printable {
    void print();
}
class Report implements Printable {
    @Override
    public void print() {
        System.out.println("Printing Report...");
    }
}
public class Library {
    public static void main(String[] args) {
        System.out.println(" BOOK DETAILS");
        Book b = new Book(101,"Java Programming","James Gosling",500);
        b.display();
        System.out.println("\n INHERITANCE");
        Student s = new Student();
        s.name = "Bob";
        s.age = 25;
        s.course = "Engineering";
        s.display();
        System.out.println("\n faculty");
        Faculty f = new Faculty();
        f.name = "Dr. Bob";
        f.age = 45;
        f.department = "Engineering";
        f.display();
        System.out.println("\n METHOD OVERLOADING");
        Area a = new Area();
        a.area(5.0);
        a.area(10, 20);
        System.out.println("\n METHOD OVERRIDING");
        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();
        v1.display();
        v2.display();
        System.out.println("\n ABSTRACTION ");
        Shape sh1 = new Circle();
        Shape sh2 = new Rectangle();
        sh1.draw();
        sh2.draw();
        System.out.println("\n INTERFACE ");
        Printable p = new Report();
        p.print();
    }
}

```

Compilation & Execution Commands

Save the file as:

Library.java

Compile the Program

javac Library.java

Run the Program

java Library

OUTPUT :

```
BOOK DETAILS
Book ID   : 101
Book Name : Java Programming
Author    : James Gosling
Price     : 500.0
```

```
INHERITANCE
Name : Bob
Age  : 25
Course : Engineering
```

```
faculty
Name : Dr. Bob
Age  : 45
Department : Engineering
```

```
METHOD OVERLOADING
Circle Area = 78.5
Rectangle Area = 200
```

```
METHOD OVERRIDING
This is a car.
This is a bike.
```

```
ABSTRACTION
Drawing a Circle
Drawing a Rectangle
```

```
INTERFACE
Printing Report...
```

```
Process finished with exit code 0
```